Lying Posture Detection

Joel Vincent

A. System Design

Motivation

Identifying the lying posture has importance as it relates to a patient's mobility and the quality of their sleep. A lightweight sensor that can automatically recognize a person's posture has value for overnight monitoring. This project implements such a system on an Arduino Nano 33 BLE Sense, using only its onboard IMU (accelerometer and gyroscope) to distinguish three postures: supine (on the back), prone (on the front), and side-lying (left or right), and to report the detected posture through a simple LED blink code.

High-Level Design

The system assumes the sensor board is mounted on a chest band, with a fixed, documented orientation relative to the body: the component (chip) side of the board faces outward, away from the chest, and the USB-C connector points toward the head. Under this convention, gravity's effect on the three accelerometer axes is different and repeatable for each of the four resting orientations, which is the physical principle the whole classifier relies on.

The project was built in four phases, matching the assignment structure:

- Phase 1: firmware to read the IMU and stream readings over Serial.
- Phase 2: data collection, recording labeled trials for each posture to a CSV file per trial.
- Phase 3: visual inspection of that data (via plots) to derive a classification rule.
- Phase 4: a real-time implementation on the board that classifies posture continuously and drives the LED -- 1 blink = supine, 2 blinks = prone, 3 blinks = side, LED off = no confident match.

Observations and Difficulties

The specific board on hand turned out to be a Nano 33 BLE Sense Rev1 rather than a Rev2. Rev1 boards carry an LSM9DS1 IMU, while Rev2 boards carry a BMI270 + BMM150 pair, and the two need different Arduino libraries (Arduino_LSM9DS1 vs. Arduino_BMI270_BMM150). Using the wrong library against Rev1 hardware led to a misleading failure mode: IMU.begin() returned true, but IMU.accelerationAvailable() never once returned true afterward, so the sketch produced a printed header and then nothing. This was resolved by writing a minimal diagnostic sketch (LED blinking) to first confirm Serial and upload were fine, then adding an un-gated print of the accelerationAvailable() flag and sample rate, which read 0 Hz. This revealed to me that I

was using the wrong library. Checking the name on the board confirmed it, and switching the include to Arduino_LSM9DS1.h resolved it immediately, since that library exposes an identical API.

Sampling Frequency

Three different rates are used across the project, since the data-collection sketch and the live classifier serve different purposes:

- Data logging (IMU_Data_Logger.ino): 5 Hz (one sample every 200 ms). Posture is a held, near-static orientation rather than fast motion, so a high rate isn't needed to characterize it.
- Live raw sampling (Posture_Detector.ino): 50 Hz (one sample every 20 ms), feeding a 200 ms moving average smoothing window (10 samples), to suppress high-frequency accelerometer noise before it reaches the classifier.
- Live classification: recomputed every 500 ms from that continuously updated smoothing buffer.

The LED itself, however, does not refresh at 500 ms. I put in a one second delay between flashes so that the user can see how many blinks are occurring

B. Experiment

Procedure

Data was collected by holding the board by hand in the orientation that represents each posture.

For each of the four postures, three trials of 20 seconds each were recorded (60 s of data per posture total), meeting the one minute of data per posture. Each trial was started with the 's' serial command described in Section A, and its output was saved to a CSV file for each posture (e.g. supine-trial.csv).

Data Collection Errors

Trials were recorded on a desk, by hand. This introduces error as I can only hold it so still. This showed up in the data: in the prone recordings, the first trial held ay of about -0.02 to -0.05, but the second and third trials both drifted to ay of about +0.28 while az remained correctly near -1.0. In other words, the board was tilted noticeably differently between trials representing the same posture. This did not break the data, because the prone rule depends only on az, but it is an example of where user error could show up in data collection.

A second, smaller artifact appeared at the boundaries of a few trials: brief, large gyroscope spikes from the natural act of picking up or repositioning the board immediately before or after a recording window. These are a small fraction of each trial's rows and did not need to be specially removed, but they are an edge case (see also Section D).

C. Algorithm

Design

At rest, gravity always contributes very close to 1 g total, split across the accelerometer's three axes according to the board's orientation. For each of the four target postures, one axis should sit near +1 g or -1 g while the other two sit near 0. Rather than assuming which axis that would be, the axis mapping used in the final code was derived from the collected trial data (Section B), which turned out to correct an initial assumption: the first draft of the classifier checked the X axis (ax) for side-lying, but inspecting the actual side-left/side-right CSVs showed the Y axis (ay) was the one that consistently read near +/-1 g, while ax stayed small in both cases.

Posture	Dominant axis	Typical value
Supine	az	+0.99
Prone	az	-1.00
Side-left	ay	+1.00
Side-right	ay	-0.99

Based on this table, classifyPosture() applies a threshold rule.

az > 0.7 means supine

az < -0.7 means prone

|ay| > 0.7 means side,

else no match.

The 0.7 g threshold was chosen with a margin under the roughly 0.97-1.0 g typically observed on the dominant axis for each posture.

A second gate uses the gyroscope: if the magnitude of angular velocity exceeds 40 deg/s, the board is considered to be in motion and the classifier reports no confident posture (UNKNOWN) rather than using an accelerometer reading that gravity plus motion could change. This was directly confirmed working during testing.

Fixed Design Bug

An early version of the LED-feedback logic used delay() calls to blink the LED and to pause between blink groups (added specifically so a person watching the board could count 1 vs. 2 vs.

3 blinks reliably, rather than seeing what looked like one continuous blink). This introduced a bug: delay() blocks the entire loop(), including the IMU-sampling code, for as long as 1.9 seconds per update (blink time plus a 1-second pause). Because that blocked duration is longer than the 500 ms classification interval, the very next classification after a blink sequence fired immediately upon returning from the delay. In practice this meant the displayed posture visibly lagged behind the board's real, current orientation by a cycle or more after being physically moved, and could occasionally show a "no posture" reading mid-transition as the averaging buffer mixed old and new samples.

The fix was to make the whole sketch non-blocking. IMU sampling and posture classification run unconditionally on every pass of loop(), gated only by millis() comparisons, never by delay(). The LED's blink-then-pause behavior is instead driven by its own small state machine (starting a group, blinking on, blinking off, pausing) that is serviced every loop() pass but never blocks it. This restored live, low-latency classification while preserving the deliberate 1-second pause between blink groups for human readability. After this change, transitions between postures were confirmed to be immediate and reliable.

Update Rate

The 0.7 g threshold, chosen with a wide margin under the roughly 0.97-1.0 g typically observed, makes the classifier tolerant of modest, realistic variation in how the board sits against the body -- small tilts away from a perfectly flat orientation do not change the classification. It is not, however, robust to a genuinely ambiguous orientation, such as holding the board at roughly 45 degrees between two defined postures; see Section D for a directly observed example of this limitation.

The live classifier recomputes a posture estimate every 500 ms from a continuously fresh 200 ms smoothing window. The LED's visible output, however, updates more slowly than that by design: each blink group (up to 900 ms for three blinks) is followed by a full second of forced darkness before the next group can start, so a person watching gets roughly one legible reading every 1.8-2.4 seconds depending on the posture shown. This is a deliberate trade of raw responsiveness for human-observability of the blink count -- a real deployed version of this system could log the 500 ms classification stream continuously in the background while only using the LED as an occasional, human-facing summary.

D. Results

Sensor Data Plots

The following figures were captured live using the Arduino IDE's built-in Serial Plotter (a small dedicated sketch printed only ax/ay/az, labeled for the plotter's legend, so all three traces share one -1..+1 g scale). For each posture, the board was held steady in that orientation until the traces settled.

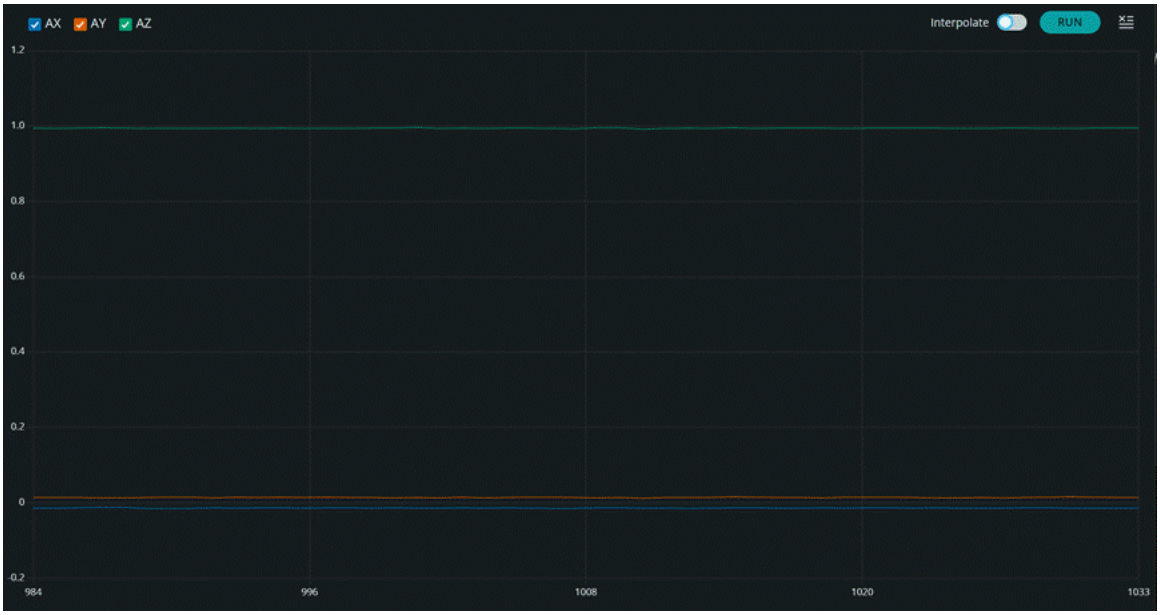


Figure 1. Supine: AZ (green) flat near +1.0 g; AX, AY near 0.

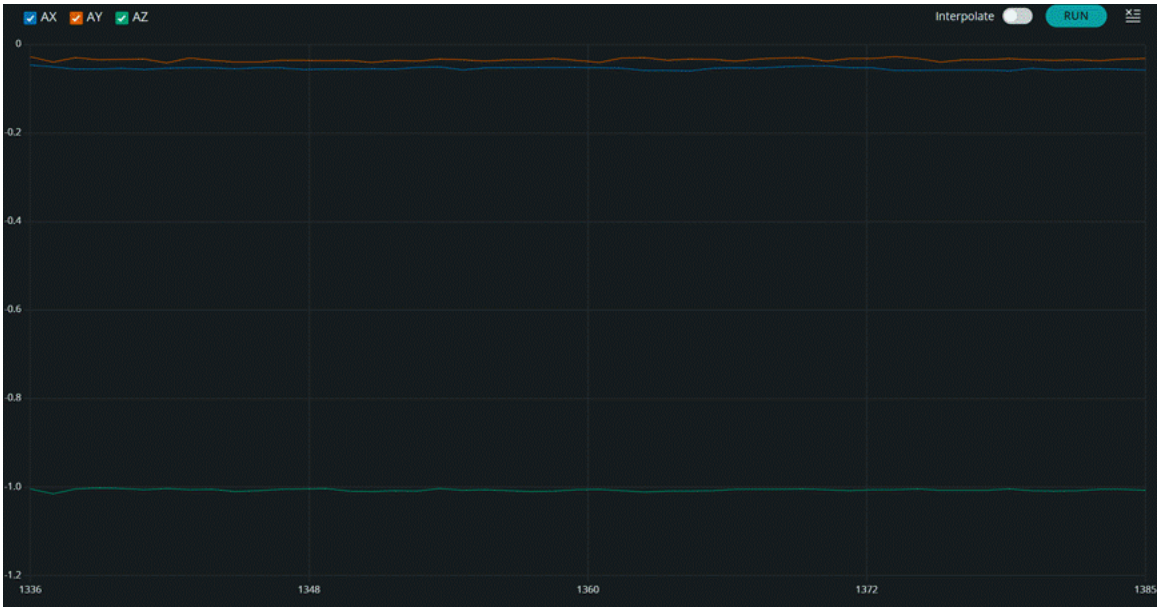


Figure 2. Prone: AZ (green) flat near -1.0 g -- a mirror image of Figure 1.

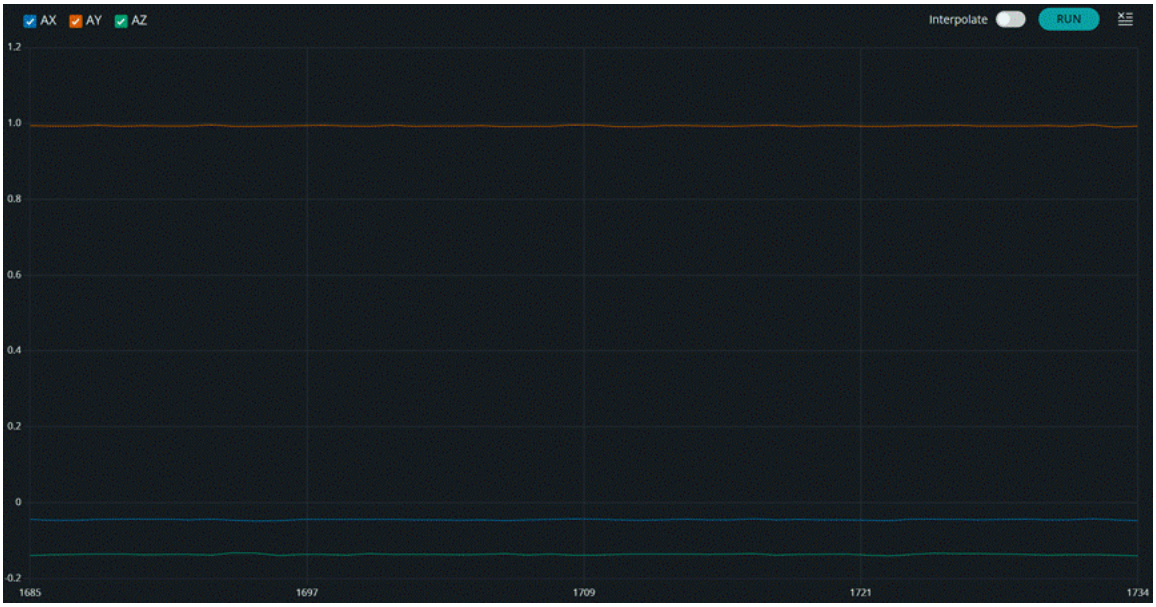


Figure 3. Side-left: AY (orange) flat near +1.0 g.

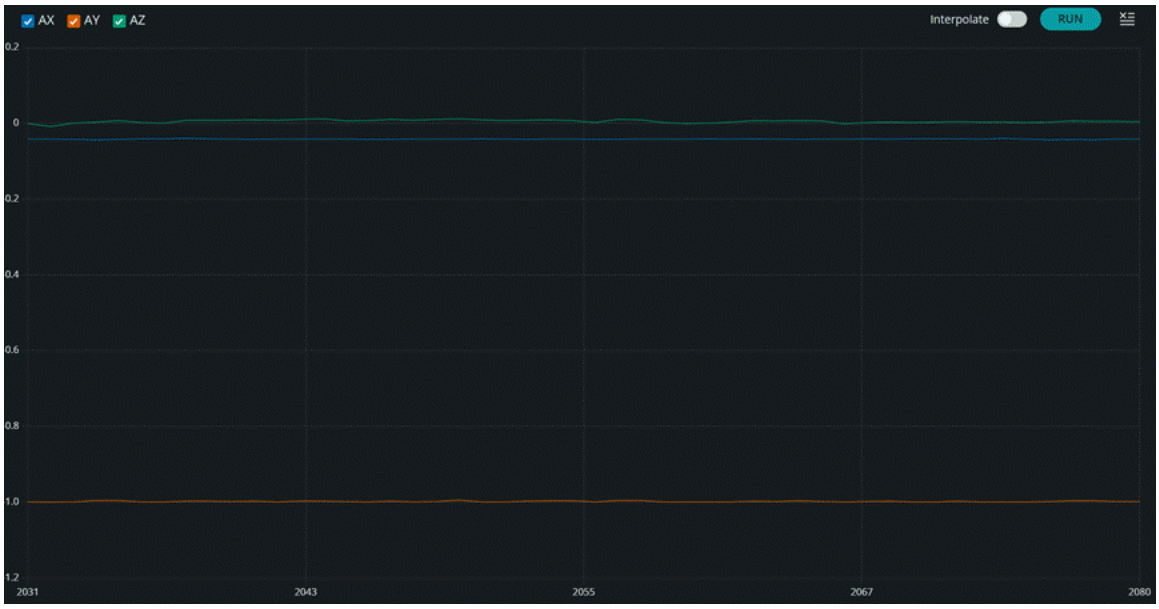


Figure 4. Side-right: AY (orange) flat near -1.0 g.

All four traces are clean and low-noise, and each posture shows the expected single dominant axis near +/-1 g with the other two near zero -- direct visual confirmation of the axis mapping used in Section C.

Accuracy

A live accuracy test was run using Posture_Detector.ino: for each of the four postures, the board was held steady and 20 consecutive classification readings (about 10 seconds, one every ~500 ms) were tallied by hand against the true, known posture.

Posture	Correct	Total	Accuracy
Supine	20	20	100%
Prone	20	20	100%
Side-left	20	20	100%
Side-right	20	20	100%
Overall	**80**	**80**	**100%**

Edge Cases

Beyond the controlled accuracy test above, two edge cases were observed and are worth documenting.

45-degree orientation. Holding the board at roughly 45 degrees, between the supine and side orientations, produced simultaneous az of about 0.6-0.75 and ay of about 0.7-0.8 -- both near or above the 0.7 g threshold at the same time. Because classifyPosture() checks az before ay, the reported posture flickered between SUPINE and SIDE from one 500 ms cycle to the next, depending on which axis's small noise fluctuation happened to cross the threshold first. An excerpt from that test is shown below:

```
ax=-0.03 ay=0.69 az=0.72 gyroMag=3.9 -> SUPINE
ax=-0.03 ay=0.75 az=0.65 gyroMag=3.8 -> SIDE
ax=-0.02 ay=0.80 az=0.59 gyroMag=3.4 -> SIDE
ax=-0.02 ay=0.80 az=0.58 gyroMag=7.4 -> SIDE
ax=-0.03 ay=0.79 az=0.60 gyroMag=9.2 -> SIDE
ax=-0.02 ay=0.73 az=0.67 gyroMag=6.5 -> SIDE
ax=-0.03 ay=0.69 az=0.71 gyroMag=14.2 -> SUPINE
ax=-0.02 ay=0.66 az=0.74 gyroMag=2.9 -> SUPINE
ax=-0.02 ay=0.67 az=0.73 gyroMag=5.2 -> SUPINE
ax=-0.02 ay=0.71 az=0.69 gyroMag=3.7 -> SIDE
ax=-0.02 ay=0.74 az=0.66 gyroMag=5.3 -> SIDE
ax=-0.02 ay=0.71 az=0.69 gyroMag=9.4 -> SIDE
ax=-0.02 ay=0.69 az=0.72 gyroMag=6.0 -> SUPINE
ax=-0.02 ay=0.68 az=0.72 gyroMag=5.8 -> SUPINE
ax=-0.02 ay=0.69 az=0.71 gyroMag=4.3 -> SUPINE
ax=-0.02 ay=0.73 az=0.67 gyroMag=4.2 -> SIDE
ax=-0.02 ay=0.76 az=0.64 gyroMag=4.8 -> SIDE
ax=-0.02 ay=0.77 az=0.62 gyroMag=8.4 -> SIDE
ax=-0.02 ay=0.74 az=0.66 gyroMag=15.1 -> SIDE
ax=-0.02 ay=0.67 az=0.74 gyroMag=34.2 -> SUPINE
ax=-0.11 ay=0.18 az=0.94 gyroMag=57.4 -> UNKNOWN
ax=-0.04 ay=0.02 az=0.99 gyroMag=5.8 -> SUPINE
```

This is a limitation of a simple, single threshold classifier. 45 degrees does not truly belong to any one of the three defined postures, so there is no single "correct" label for the classifier to settle on, and the fixed if/else check order (az before ay) breaks the tie inconsistently rather than deterministically. Rather than patching this with an ad-hoc tie-break rule, it was decided to document it as-is: it is an authentic boundary case, not a defect, and any fix would still be arbitrary at exactly 45 degrees.

Note also the sixth line above (gyroMag = 57.4 deg/s -> UNKNOWN): this is the stillness gate working exactly as designed, rejecting a reading taken while the board was actively being adjusted rather than reporting a potentially bogus classification.

Prone orientation drift between trials. As described in Section B, two of the three prone trials drifted to ay of about +0.28 instead of the first trial's ay of about -0.03, while az remained a consistent -1.0. This did not cause a misclassification (the rule depends only on az for prone),

but it demonstrates real intra-class variability introduced simply by re-holding a posture by hand across separate trials.

E. Discussion

Summary

The final system reliably classifies all four target lying postures (supine, prone, side-left, side-right) in real time from IMU data alone, using a lightweight, hand-derived threshold rule rather than a trained model. In controlled testing it achieved 100% accuracy (80/80 readings) across all four postures, and it correctly declines to classify during active motion via a gyroscope-based stillness gate. Its known limitation is at genuinely ambiguous orientations between two defined postures (e.g., 45 degrees), where it flickers between the two nearest labels rather than settling on one -- an expected consequence of using independent per-axis thresholds rather than a more holistic decision rule.

Difficulties Encountered

- An unexpected library mismatch (Rev1 vs. Rev2) cost a lot of time early into the project, and was only resolved by building small test cases and consulting LLM's asking for help on why it wasn't working.
- The Arduino IDE's Serial Monitor had no dependable way to save or reliably select large amounts of streamed text, which required redesigning the data-logger sketch around a bounded, serial monitor window instead of a continuous stream.
- A subtle timing bug (blocking delay() calls silently freezing IMU sampling during LED feedback) caused the live display to visibly lag behind the board's actual orientation; finding and fixing it required recognizing that the blink logic itself, not the classification logic, was the source of the delay.
- The classifier's inherent ambiguity at boundary angles like 45 degrees is a limitation of the threshold-based design chosen here.

Future Improvements

- Repeat the accuracy test across multiple sessions, days, and ideally multiple people, to get a more realistic accuracy estimate than a single best-case session.
- Validate on an actual worn chest band rather than a hand-held simulation, since a worn sensor's orientation is constrained by the body in ways a hand-held test cannot fully replicate.
- Replace the fixed if/else axis priority with a more principled decision rule for ambiguous readings -- for example, picking whichever axis has the larger magnitude when more than one exceeds threshold, or comparing against reference gravity vectors for all six board orientations (a nearest-neighbor style classifier) instead of independent per-axis thresholds.

- Consider per-user or adaptive thresholds, since differences in body shape and how snugly a chest band fits could shift the effective resting gravity vector slightly from one wearer to the next.

Did the Real-Time Approach Work?

Yes. After the non-blocking rewrite, the system responded to posture changes immediately and reliably, correctly rejected readings taken during active motion, and achieved perfect accuracy across all four postures in controlled testing. Its one demonstrated failure mode -- flickering between two labels at a genuinely ambiguous 45-degree orientation -- is an honest reflection of the classifier's simplicity rather than an implementation defect, and is documented here rather than papered over.